TA9: Survival and Missing Data

09/11/2026

The Cancer and Leukemia Group B dataset

Study CALGB 8541 (Example 11.1 from RLO)

The Cancer and Leukemia Group B (CALGB) conducted a randomized clinical trial to compare the benefits of three dose regimen intensities (high, moderate, and low) of a three-drug combination to treat non-metastatic breast cancer. The study, CALGB 8541, randomized over 1,500 women to one of the three treatment regimens.

The data set contains several variables:
`seqno, study, arm, er, pgr, npn, tsizecm, premeno, survyrs, survstat, dfsyr` , and `dfsstat` .

`survyrs` and `survstat` , respectively, contain the survival time in years and its censoring status (0 if uncensored time of death, 1 if right censored). Note that in the notation of Sections 11.1.2 and 11.1.3, this variable represents δ_1 and not δ itself.

The variables `dfsyrs` and `dfsstat` are similar, except they refer to disease-free survival. This means

if `dfsstat` =0, then `dfsyrs` is the time at which either death or disease relapse or recurrence occurred;

if `dfsstat` =1, then dfsyrs is the time at which the patient was last known to be alive without relapse or recurrence of disease (but the patient's disease or survival status is not available beyond this point).

There are 519, 517, and 513 subjects in arms 1, 2, and 3, respectively. Of these, right censored observations number 265, 231, and 244 for the survival outcome and 224, 188, and 197 for disease-free survival. The maximum follow-up time is about 20 years in each arm.

Focus only on arm 1, we consider a parametric model

Let T denote the dime of death (`survyrs`). The joint likelihood can be derived by assuming

$$T_1, \dots, T_n \mid f \sim f(t)$$

For the moment f is a generic density function

When observations are censored, evaluation of the likelihood is impossible (of course). Let C_i denote the random censoring time of observation i, we assume that

1. Censoring is independent of the event of interest
2. $P(C \leq c)$ does not depend on the θ [non-informative censoring]

Hence

$$T_i, C_i \mid f, g \sim f(t)g(c)$$

We rewrite the model in terms of (y_i, δ_i) such that $\delta = 0$ iff $Y = C$ and $\delta = 1$ iff $Y = T$.

We introduce the hazard and survival functions: $f(t) = h(t)S(t)$ and write the likelihood as

$$L\left(\{y_i, \delta_i\}\right) \propto \prod f(y_i)^{\delta_i} S(y_i)^{1-\delta_i}$$

```r
# Import modules
library(posterior)
library(bayesplot)
library(ggplot2)

# Import functions
library(cmdstanr)

# Create ./stan folder if does not exists
if (!dir.exists("./stan")) {
    dir.create("./stan")
}
```

```r
df <- read.csv("CALGB8541.csv")
df <- df[df$arm == 1, ] # restrict analysis to first arm
df <- df[sample(nrow(df)), ]
y <- df$survyrs
delta <- as.integer(1 - df$survstat)
```

```r
cols <- c("steelblue", "orange")
# steelbleu = censored - orange = observed
plot(seq_along(y), y, col = cols[delta + 1], pch = 19, xlab = '')
```

For **prior elicitation**, we can use some statistics about Breast cancer available here (https://www.cancer.net/cancer-types/breast-cancer/statistics). From this page, we get the following information: - 5 years survival: 90% - 10 years survival: 84%

Exponential model

$$f(t|\theta) = \theta e^{-\theta t}$$

$$\theta \sim \text{Gamma}(a, b)$$

The quantile function of the exponential is

$$t_p = \frac{-log(1-p)}{\theta}$$

we solve for $t_{0.1} = 5$ and $t_{0.16} = 10$ setting $\theta = \hat{\theta} = a/b$

```r
b = - 5 / log(0.9)
a = 0.5 * log(0.84) / log(0.9)
cat(a, b)
```

```
## 0.8274133 47.45611
```

```r
get_exp_posterior <- function(y, delta, a, b) {
    return(c(sum(delta) + a, sum(y) + b))
}

post_params <- get_exp_posterior(y, delta, a, b)
post_samples <- rgamma(100, shape = post_params[1], rate = post_params[2])
```

```
t_range <- seq(0, 20, length.out = 1000)
surv <- exp(-outer(post_samples, t_range))
plot(t_range, surv[1, ], type = "l", col = adjustcolor("steelblue", alpha.f = 0.3), ylim = c(0, 1.1), x
lab = "", ylab = "")
for (i in 2:nrow(surv)) {
    lines(t_range, surv[i, ], col = adjustcolor("steelblue", alpha.f = 0.3))
}
```

Weibull model

$$T \sim Wieb(\alpha, \lambda) \iff T^\alpha \sim Exp(\lambda)$$

$$f(t \mid \alpha, \lambda) = \lambda \alpha t^{\alpha-1} \exp(-\lambda t^\alpha) \mathbb{I}[t > 0]$$

How to set priors for α and λ?

First, we set good guesses: for the Weibull distribution we know

$$1 - q = P(T > t_q) = \exp(-\lambda t_q^\alpha)$$

and we know that $P(T > 5) = 0.9$, $P(T > 10) = 0.84$ from previous studies.

So that we solve

$$\begin{cases} 0.9 = \exp(-\lambda 5^\alpha) \\ 0.84 = \exp(-\lambda 10^\alpha) \end{cases}$$

```
obj_func <- function(x) {
    alpha <- exp(x[1])
    lamb <- exp(x[2])
    return((log(0.9) + lamb * 5^alpha)^2 + (log(0.84) + lamb * 10^alpha)^2)
}

out <- optim(par = c(1, -11), fn = obj_func)
vals <- exp(out$par)
vals
```

```
## [1] 0.72667146 0.03271757
```

```
# We compute the expected value for these parameters

1.0 / vals[2] * gamma(1 + 1.0 / vals[1])
```

```
## [1] 37.38501
```

Watch out!

Stan uses a different parametrization

$$f(t \mid \alpha, \sigma) = \frac{\alpha}{\sigma} \left(\frac{t}{\sigma} \right)^{\alpha-1} \exp\left(-\frac{t^\alpha}{\sigma} \right) \mathbb{I}[t > 0]$$

So that $\sigma = (1/\lambda)^\alpha$

```
obj_func <- function(x) {
    alpha <- exp(x[1])
    sigma <- exp(x[2])
    return((log(0.9) + (5 / sigma)^alpha)^2 + (log(0.84) + (10 / sigma)^alpha)^2)
}

out <- optim(par = c(-1, -1), fn = obj_func)
vals <- exp(out$par)
vals
```

```
## [1]   0.7295992 109.5641136
```

Given that σ is very far from 0, a log-Normal or a Gamma would make little difference. While for α that is pretty close to 0, a log-Normal is usually better.

The median of the log-normal distribution is $\exp(\mu)$, so we set

$$\alpha \sim \log N(\log 0.726, s_1^2)$$

$$\sigma \sim \log N(\log 109.56, s_2^2)$$

To set s_1^2 and s_2^2 observe that the mode of a log-normal is $\exp(\mu_j - s_j^2)$ so that large values of s_j^2 force the mode near to zero.

We set $s_1 = 1.0$ and $s_2 = 10$.

```
weib_censored <- "
data {
    int<lower=0> n_obs;
    int<lower=0> n_cens;
    array[n_obs] real y_obs;
    array[n_cens] real y_cens;
}

parameters {
    real<lower=0> alpha;
    real<lower=0> sigma;
}

model {
    y_obs ~ weibull(alpha, sigma);
    for (i in 1:n_cens)
        target += weibull_lccdf(y_cens[i] | alpha, sigma);

    alpha ~ lognormal(-0.3202, 1.0);
    sigma ~ lognormal(4.70, 10.0);
}
"

# Write stan model to file
stan_file <- "./stan/weib_censored.stan"
writeLines(weib_censored, stan_file)

# Compile stan model
weib_censored <- cmdstan_model(stan_file)
```

```
y_obs <- y[which(delta == 1)]
y_cens <- y[which(delta == 0)]
```

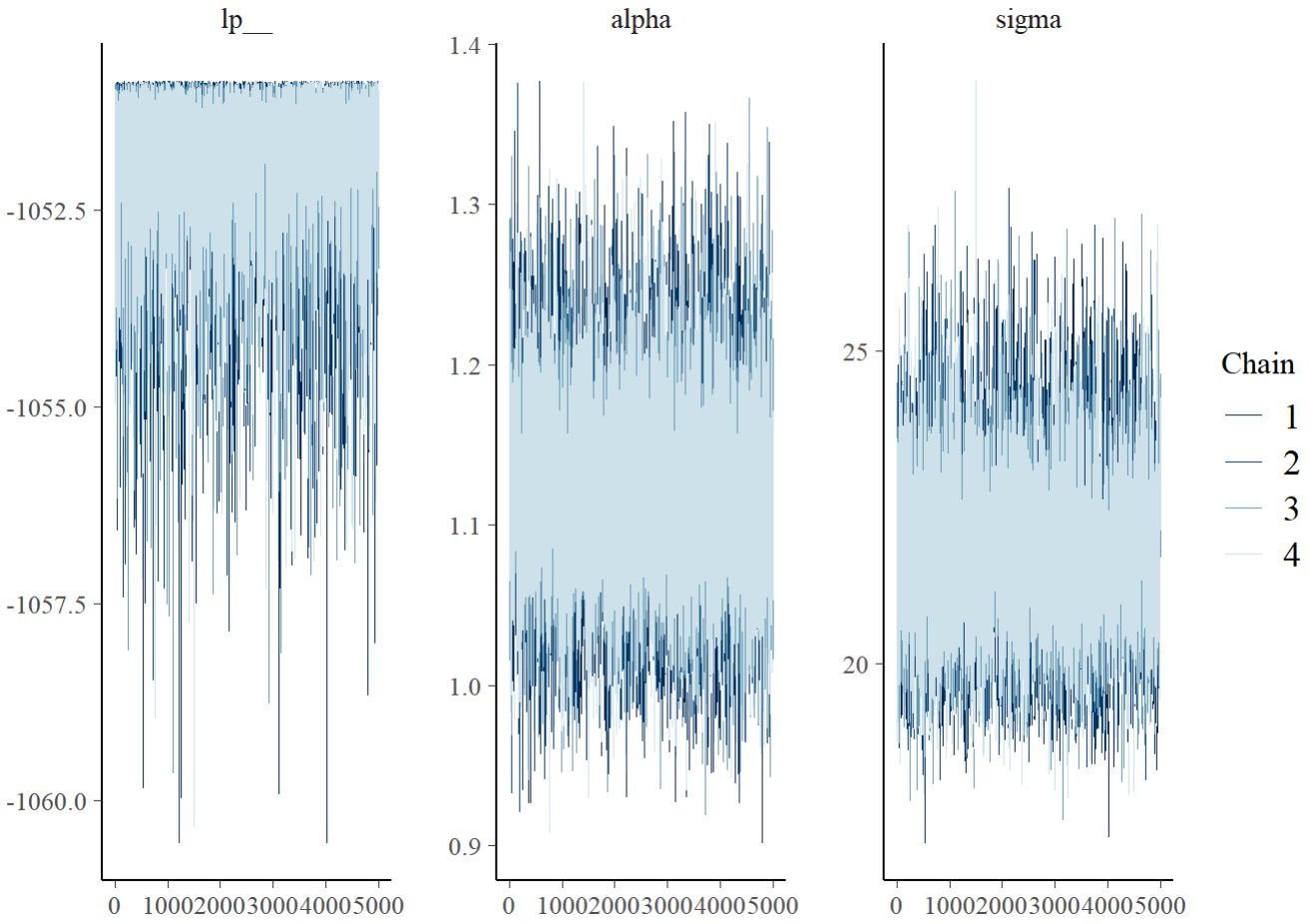
```
weib_data <- list(
    n_obs = length(y_obs),
    n_cens = length(y_cens),
    y_obs = y_obs,
    y_cens = y_cens
)

weib_censored_fit <- weib_censored$sample(
    data = weib_data,
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 5000,
    show_exceptions = FALSE,
    refresh = 1000
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 1 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 1 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 2 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 3 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 3 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 4 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 4 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 4 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 2 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 3 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 1 finished in 0.9 seconds.
## Chain 2 finished in 0.9 seconds.
## Chain 3 finished in 0.9 seconds.
## Chain 4 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 4 finished in 1.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 0.9 seconds.
## Total execution time: 1.2 seconds.
```

```
posterior_draws_weib <- posterior::as_draws_df(weib_censored_fit)
```

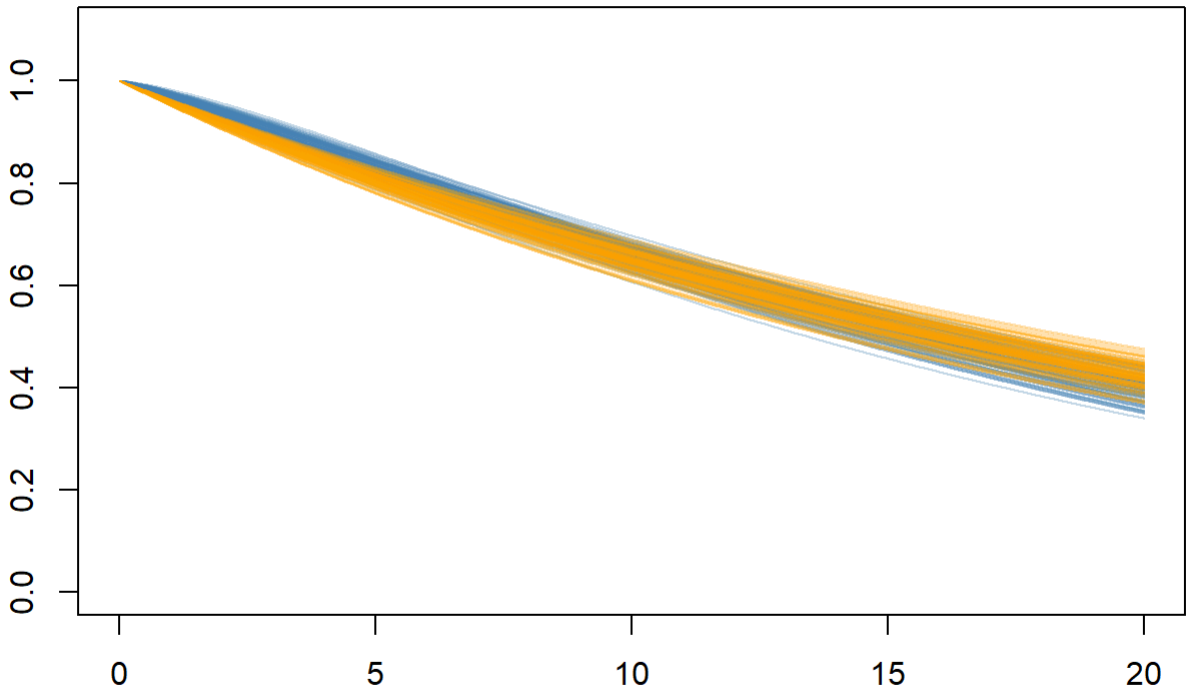
```
mcmc_trace(posterior_draws_weib)
```




```
alphas <- posterior_draws_weib$alpha
sigmas <- posterior_draws_weib$sigma

t_range <- seq(0, 20, length.out = 1000)

plot(NA, xlim = c(0, 20), ylim = c(0, 1.1), xlab = "", ylab = "")
# weibull survival functions
for (i in 1:100) {
    lines(t_range, exp(-(t_range / sigmas[i])^alphas[i]), col = adjustcolor("steelblue", alpha.f = 0.
3))
}
# exponential survival
surv <- exp(-outer(post_samples, t_range))
for (i in 1:length(post_samples)) {
    lines(t_range, surv[i, ], col = adjustcolor("orange", alpha.f = 0.3))
}
```



Let's take it Hierarchical

We consider the same dataset as before, but take into account different arms

j: study arm. i: patient's index (inside the arm)

$$
\begin{aligned}
L\left(\{y_{j,i}, \delta_{j,i}\}_{j,i} \mid \theta_1, \ldots, \theta_J\right) &= \prod f(y_{j,i} \mid \theta_j)^{\delta_{j,i}} S(y_{j,i})^{1-\delta_{j,i}} \\
\theta_j &= (\alpha_j, \sigma_j) \\
\alpha_1, \ldots, \alpha_J \mid \alpha_0 &\sim \log N(\alpha_0, 1.0) \\
\sigma_1, \ldots, \sigma_J \mid \sigma_0 &\sim \log N(\sigma_0, 10.0) \\
\alpha_0 &\sim \log N(\log 0.726, 1.0) \\
\sigma_0 &\sim \log N(\log 109.56, 10.0)
\end{aligned}
$$

where f and S represent the cdf and survival function of a Weibull distribution, respectively.

```
weib_hier <- "
data {
    int<lower=0> n_groups;
    int<lower=0> n_tot;
    array[n_tot] real y;
    array[n_tot] int<lower=0, upper=n_groups> idx2group;
    array[n_tot] int<lower=0, upper=1> observed;
}

parameters {
    array[n_groups] real<lower=0> alpha;
    array[n_groups] real<lower=0> sigma;
    real<lower=0> alpha0;
    real<lower=0> sigma0;
}


model {
    for (i in 1:n_tot) {
        if (observed[i] == 1) {
            y[i] ~ weibull(alpha[idx2group[i]], sigma[idx2group[i]]);
        } else {
            target += weibull_lccdf(
                y[i] | alpha[idx2group[i]], sigma[idx2group[i]]);
        }
    }

    alpha ~ lognormal(alpha0, 1.0);
    sigma ~ lognormal(sigma0, 10.0);

    alpha0 ~ lognormal(-0.3202, 1.0);
    sigma0 ~ lognormal(4.70, 10.0);
}
"

# Write stan model to file
stan_file <- "./stan/weib_hier.stan"
writeLines(weib_hier, stan_file)

# Compile stan model
weib_hier <- cmdstan_model(stan_file)
```

```
df <- read.csv("CALGB8541.csv")
y <- df$survyrs
delta <- 1 - df$survstat
arm <- df$arm

weib_hier_data <- list(
    n_groups = length(unique(arm)),
    n_tot = length(y),
    y = y,
    idx2group = arm,
    observed = delta
)

weib_hier_fit <- weib_hier$sample(
    data = weib_hier_data,
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 5000,
    refresh = 1000,
    show_exceptions = FALSE
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 3 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 2 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 2 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 4 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 4 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 1 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 3 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 4 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 1 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 1 finished in 10.8 seconds.
## Chain 3 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 3 finished in 11.2 seconds.
## Chain 4 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 4 finished in 11.5 seconds.
## Chain 2 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 2 finished in 11.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 11.3 seconds.
## Total execution time: 11.8 seconds.
```

```
## Warning: 1811 of 20000 (9.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.
```

```
weib_hier_fit
```

```
##  variable     mean   median   sd  mad       q5      q95 rhat ess_bulk ess_tail
##  lp__     -3267.59 -3267.29 2.01 1.88 -3271.35 -3264.90 1.00     7013    10618
##  alpha[1]     1.13     1.13 0.06 0.06     1.03     1.24 1.00    11298    11385
##  alpha[2]     1.00     1.00 0.05 0.05     0.91     1.09 1.00     9978    10196
##  alpha[3]     1.12     1.12 0.06 0.06     1.02     1.22 1.00    10548    10290
##  sigma[1]    21.78    21.70 1.37 1.36    19.67    24.15 1.00    11580    11121
##  sigma[2]    18.50    18.44 1.16 1.16    16.72    20.50 1.00     9859     9701
##  sigma[3]    20.19    20.12 1.21 1.19    18.32    22.28 1.00     9922    10454
##  alpha0       0.44     0.38 0.29 0.26     0.10     1.01 1.00    11072     9123
##  sigma0       1.01     0.03 2.19 0.04     0.00     6.02 1.00     5653     6259
```

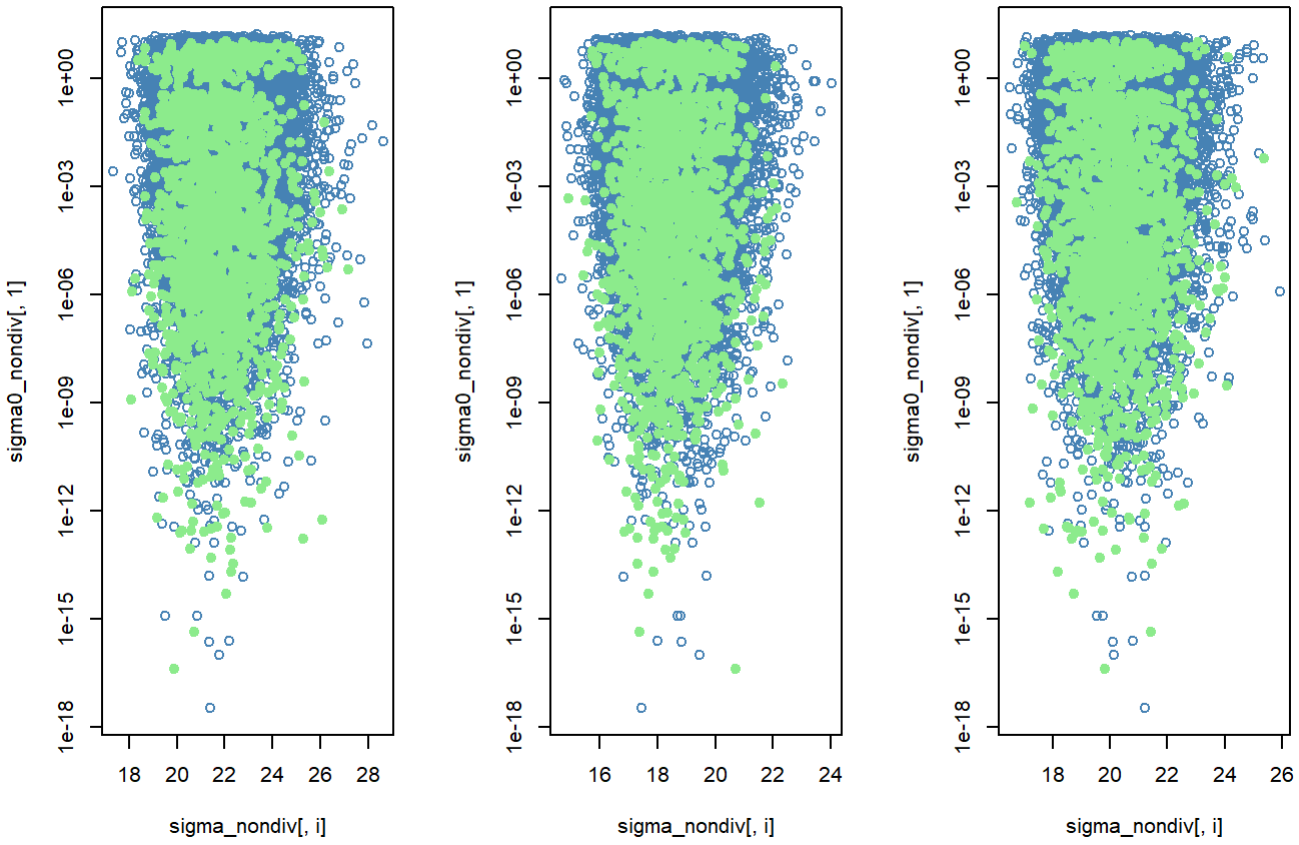
```
split_diverging <- function(fit, var_name) {
    diag_mat <- as.matrix(fit$sampler_diagnostics(format = "matrix"))
    div <- diag_mat[, "divergent__"] == 1
    vals_mat <- as.matrix(fit$draws(var_name, format = "matrix"))
    return(list(vals_mat[div, , drop = FALSE], vals_mat[!div, , drop = FALSE]))
}
```

```
par(mfrow = c(1, 3))

sigma_res <- split_diverging(weib_hier_fit, "sigma")
sigma_div <- sigma_res[[1]]
sigma_nondiv <- sigma_res[[2]]

sigma0_res <- split_diverging(weib_hier_fit, "sigma0")
sigma0_div <- sigma0_res[[1]]
sigma0_nondiv <- sigma0_res[[2]]

for (i in 1:3) {
    plot(sigma_nondiv[, i], sigma0_nondiv[, 1], col = "steelblue", log = "y")
    points(sigma_div[, i], sigma0_div[, 1], col = "lightgreen", pch = 19)
}
```



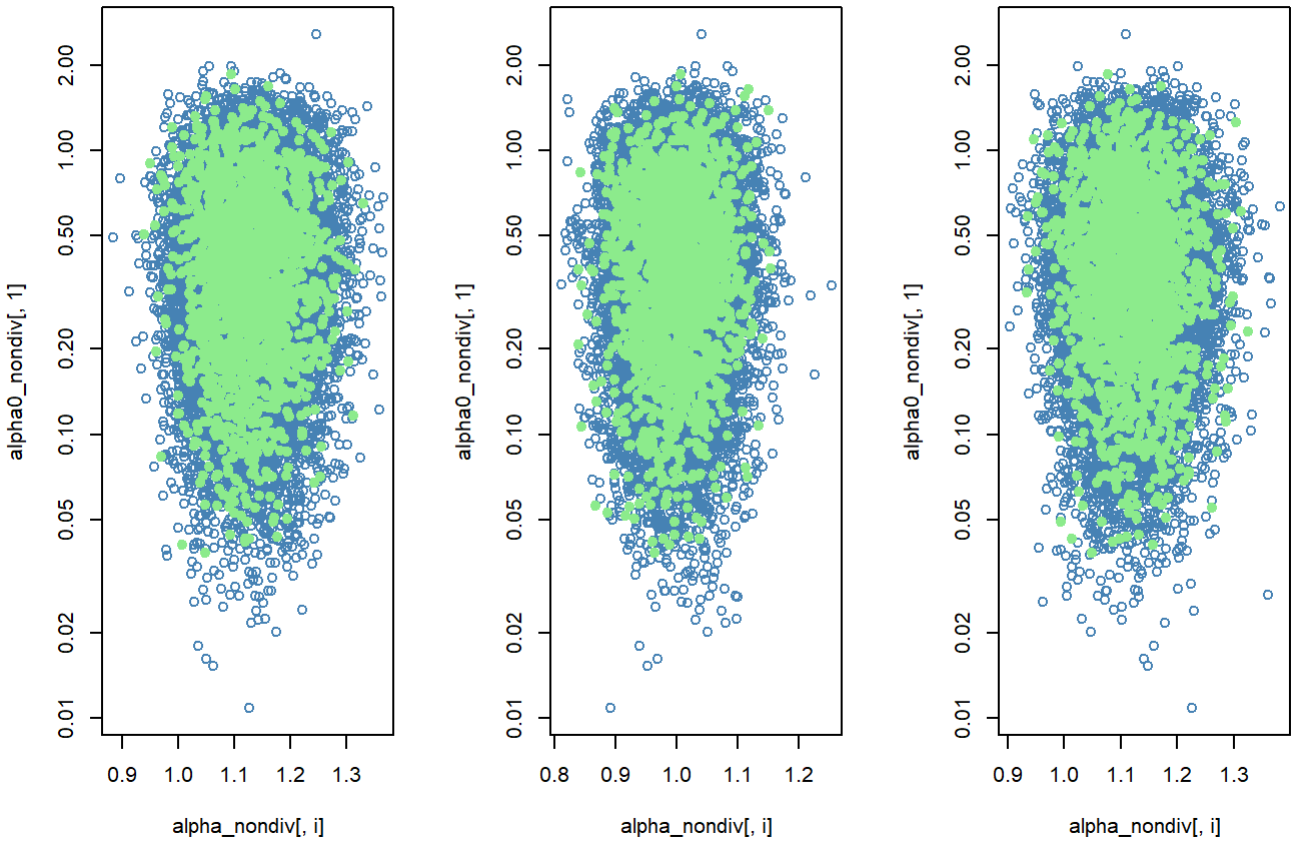

```
par(mfrow = c(1, 1))
```

```
par(mfrow = c(1, 3))

alpha_res <- split_diverging(weib_hier_fit, "alpha")
alpha_div <- alpha_res[[1]]
alpha_nondiv <- alpha_res[[2]]

alpha0_res <- split_diverging(weib_hier_fit, "alpha0")
alpha0_div <- alpha0_res[[1]]
alpha0_nondiv <- alpha0_res[[2]]

for (i in 1:3) {
    plot(alpha_nondiv[, i], alpha0_nondiv[, 1], col = "steelblue", log = "y")
    points(alpha_div[, i], alpha0_div[, 1], col = "lightgreen", pch = 19)
}
```




```
par(mfrow = c(1, 1))
```

The problem may come come from σ_0 and/or α_0, which take values that are too small

Let us try to reduce the variances of σ_0 to 3^2 and of α's and σ's to 1.5^2

```
weib_hier_reparam <- "
data {
    int<lower=0> n_groups;
    int<lower=0> n_tot;
    array[n_tot] real y;
    array[n_tot] int<lower=0, upper=n_groups> idx2group;
    array[n_tot] int<lower=0, upper=1> observed;
}

parameters {
    array[n_groups] real alpha_raw;
    array[n_groups] real sigma_raw;
    real alpha0_raw;
    real sigma0_raw;
}

transformed parameters {
    real alpha0;
    real sigma0;
    array[n_groups] real<lower=0> alpha;
    array[n_groups] real<lower=0> sigma;

    alpha0 = exp(alpha0_raw * 1.0 - 0.3202);
    sigma0 = exp(sigma0_raw * 3.0 + 4.70);

    for (i in 1:n_groups) {
        alpha[i] = exp(alpha0 + 1.5 * alpha_raw[i]);
        sigma[i] = exp(sigma0 + 1.5 * sigma_raw[i]);
    }

}

model {
    for (i in 1:n_tot) {
        if (observed[i] == 1) {
            y[i] ~ weibull(alpha[idx2group[i]], sigma[idx2group[i]]);
        } else {
            target += weibull_lccdf(
                y[i] | alpha[idx2group[i]], sigma[idx2group[i]]);
        }
    }
    alpha_raw ~ normal(0.0, 1.0);
    sigma_raw ~ normal(0.0, 1.0);
    alpha0_raw ~ normal(0.0, 1.0);
    sigma0_raw ~ normal(0.0, 1.0);
}
"

stan_file <- "./stan/weib_hier_reparam.stan"

writeLines(weib_hier_reparam, stan_file)

weib_hier_reparam <- cmdstan_model(stan_file)
```

```
## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
_array(stan::rng_t&, std::vector<double>&, std::vector<int>&, std::vector<double>&, bool, bool, std::os
tream*) const [with M = weib_hier_reparam_model_namespace::weib_hier_reparam_model; stan::rng_t = boos
t::random::mixmax_engine<17, 36, 0>; std::ostream = std::basic_ostream<char>]' was hidden [-Woverloaded
-virtual=]
##   205 |   void write_array(stan::rng_t& rng, std::vector<double>& theta,
##       |        ^~~~~~~~~~
```

```
## C:/Users/gbelt/AppData/Local/Temp/RtmpsNFwxG/model-7600e003b0a.hpp:653:3: note:   by 'void weib_hier
_reparam_model_namespace::weib_hier_reparam_model::write_array(RNG&, Eigen::Matrix<double, -1, 1>&, Eig
en::Matrix<double, -1, 1>&, bool, bool, std::ostream*) const'
##   653 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
_array(stan::rng_t&, Eigen::VectorXd&, Eigen::VectorXd&, bool, bool, std::ostream*) const [with M = wei
b_hier_reparam_model_namespace::weib_hier_reparam_model; stan::rng_t = boost::random::mixmax_engine<17,
36, 0>; Eigen::VectorXd = Eigen::Matrix<double, -1, 1>; std::ostream = std::basic_ostream<char>]' was h
idden [-Woverloaded-virtual=]
##   136 |   void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##       |        ^~~~~~~~~~
## C:/Users/gbelt/AppData/Local/Temp/RtmpsNFwxG/model-7600e003b0a.hpp:653:3: note:   by 'void weib_hier
_reparam_model_namespace::weib_hier_reparam_model::write_array(RNG&, Eigen::Matrix<double, -1, 1>&, Eig
en::Matrix<double, -1, 1>&, bool, bool, std::ostream*) const'
##   653 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
_prob(Eigen::VectorXd&, std::ostream*) const [with M = weib_hier_reparam_model_namespace::weib_hier_rep
aram_model; Eigen::VectorXd = Eigen::Matrix<double, -1, 1>; std::ostream = std::basic_ostream<char>]' w
as hidden [-Woverloaded-virtual=]
##    91 |   inline double log_prob(Eigen::VectorXd& theta,
##       |                 ^~~~~~~~
## C:/Users/gbelt/AppData/Local/Temp/RtmpsNFwxG/model-7600e003b0a.hpp:688:3: note:   by 'T_ weib_hier_r
eparam_model_namespace::weib_hier_reparam_model::log_prob(Eigen::Matrix<T, -1, 1>&, std::ostream*) cons
t'
##   688 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |
```

```
## ^~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
tp<M>::log_prob(std::vector<stan::math::var_value<double>, std::allocator<stan::math::var_value<double>
> >&, std::vector<int>&, std::ostream*) const [with M = weib_hier_reparam_model_namespace::weib_hier_re
param_model; stan::math::var = stan::math::var_value<double>; std::ostream = std::basic_ostream<char>]'
was hidden [-Woverloaded-virtual=]
##   159 |   inline math::var log_prob(std::vector<math::var>& theta,
##       |                     ^~~~~~~~
## C:/Users/gbelt/AppData/Local/Temp/RtmpsNFwxG/model-7600e003b0a.hpp:688:3: note:   by 'T_ weib_hier_r
eparam_model_namespace::weib_hier_reparam_model::log_prob(Eigen::Matrix<T, -1, 1>&, std::ostream*) cons
t'
##   688 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crt
p<M>::log_prob(Eigen::Matrix<stan::math::var_value<double>, -1, 1>&, std::ostream*) const [with M = wei
b_hier_reparam_model_namespace::weib_hier_reparam_model; stan::math::var = stan::math::var_value<double
>; std::ostream = std::basic_ostream<char>]' was hidden [-Woverloaded-virtual=]
##    96 |   inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##       |                     ^~~~~~~~
## C:/Users/gbelt/AppData/Local/Temp/RtmpsNFwxG/model-7600e003b0a.hpp:688:3: note:   by 'T_ weib_hier_r
eparam_model_namespace::weib_hier_reparam_model::log_prob(Eigen::Matrix<T, -1, 1>&, std::ostream*) cons
t'
##   688 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::lo
g_prob(std::vector<double>&, std::vector<int>&, std::ostream*) const [with M = weib_hier_reparam_model_
namespace::weib_hier_reparam_model; std::ostream = std::basic_ostream<char>]' was hidden [-Woverloaded-
```

```
## virtual=]
##   154 |   inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##       |                 ^~~~~~~~
## C:/Users/gbelt/AppData/Local/Temp/RtmpsNFwxG/model-7600e003b0a.hpp:688:3: note:   by 'T_ weib_hier_r
eparam_model_namespace::weib_hier_reparam_model::log_prob(Eigen::Matrix<T, -1, 1>&, std::ostream*) cons
t'
##   688 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
```

```
stan_data <- list(
    n_groups = length(unique(arm)),
    n_tot = length(y),
    y = y,
    idx2group = arm,
    observed = delta
)

weib_hier_reparam_fit <- weib_hier_reparam$sample(
    data = stan_data,
    chains = 4,
    parallel_chains = 4,
    adapt_delta = 0.9,
    iter_warmup = 1000,
    iter_sampling = 1000,
    refresh = 500,
    show_exceptions = FALSE,
    seed = 234
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 2 Rejecting initial value:
```

```
## Chain 2   Error evaluating the log probability at the initial value.
```

```
## Chain 2 Exception: weibull_lccdf: Scale parameter is inf, but must be positive finite! (in 'C:/User
s/gbelt/AppData/Local/Temp/RtmpsNFwxG/model-7600e003b0a.stan', line 38, column 12 to line 39, column 6
5)
```

```
## Chain 2 Rejecting initial value:
```

```
## Chain 2   Error evaluating the log probability at the initial value.
```

```
## Chain 2 Exception: weibull_lccdf: Scale parameter is inf, but must be positive finite! (in 'C:/User
s/gbelt/AppData/Local/Temp/RtmpsNFwxG/model-7600e003b0a.stan', line 38, column 12 to line 39, column 6
5)
```

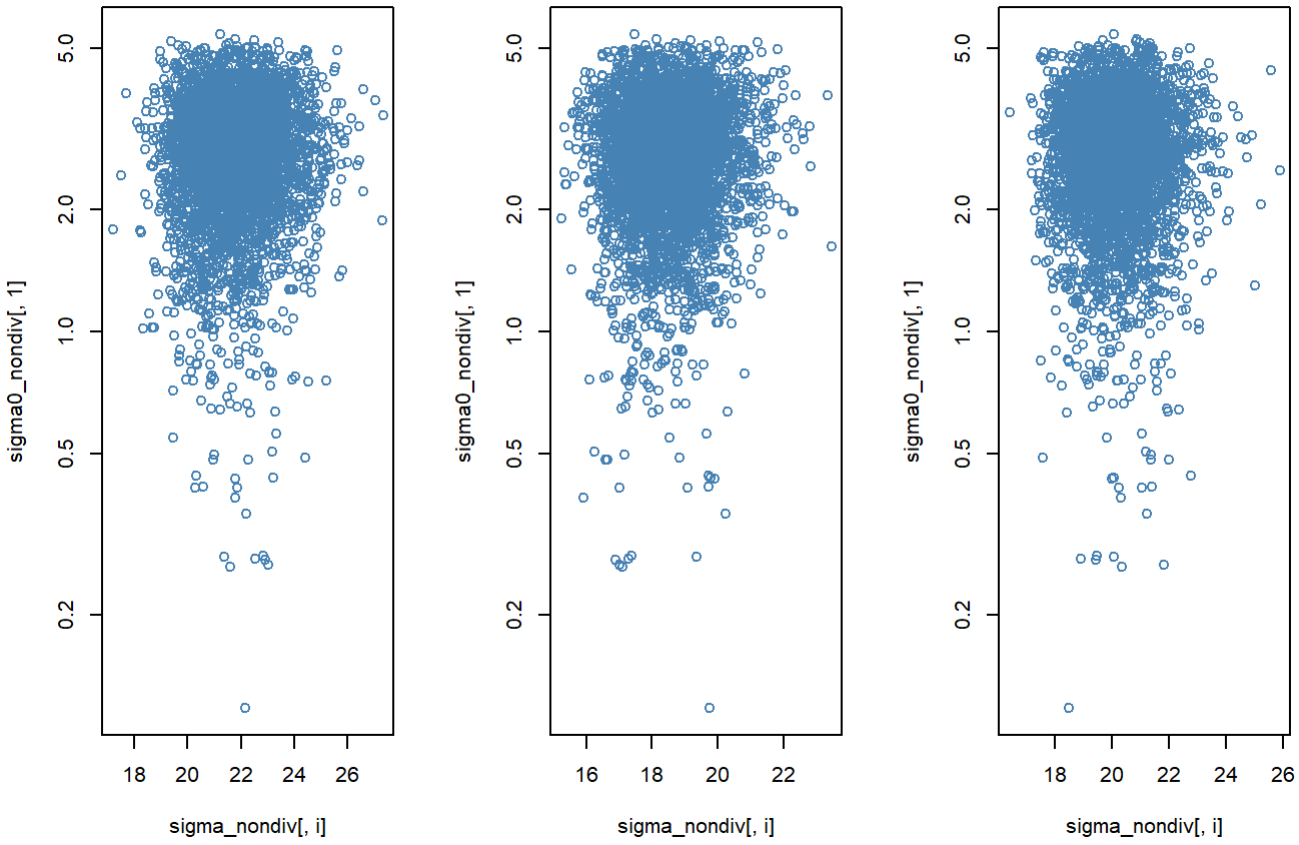
```
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 35.5 seconds.
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 37.7 seconds.
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 44.0 seconds.
## Chain 3 finished in 44.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 40.3 seconds.
## Total execution time: 44.2 seconds.
```

```
par(mfrow = c(1, 3))

sigma_res <- split_diverging(weib_hier_reparam_fit, "sigma")
sigma_div <- sigma_res[[1]]
sigma_nondiv <- sigma_res[[2]]

sigma0_res <- split_diverging(weib_hier_reparam_fit, "sigma0")
sigma0_div <- sigma0_res[[1]]
sigma0_nondiv <- sigma0_res[[2]]

for (i in 1:3) {
    plot(sigma_nondiv[, i], sigma0_nondiv[, 1], col = "steelblue", log = "y")
    points(sigma_div[, i], sigma0_div[, 1], col = "lightgreen", pch = 19)
}
```



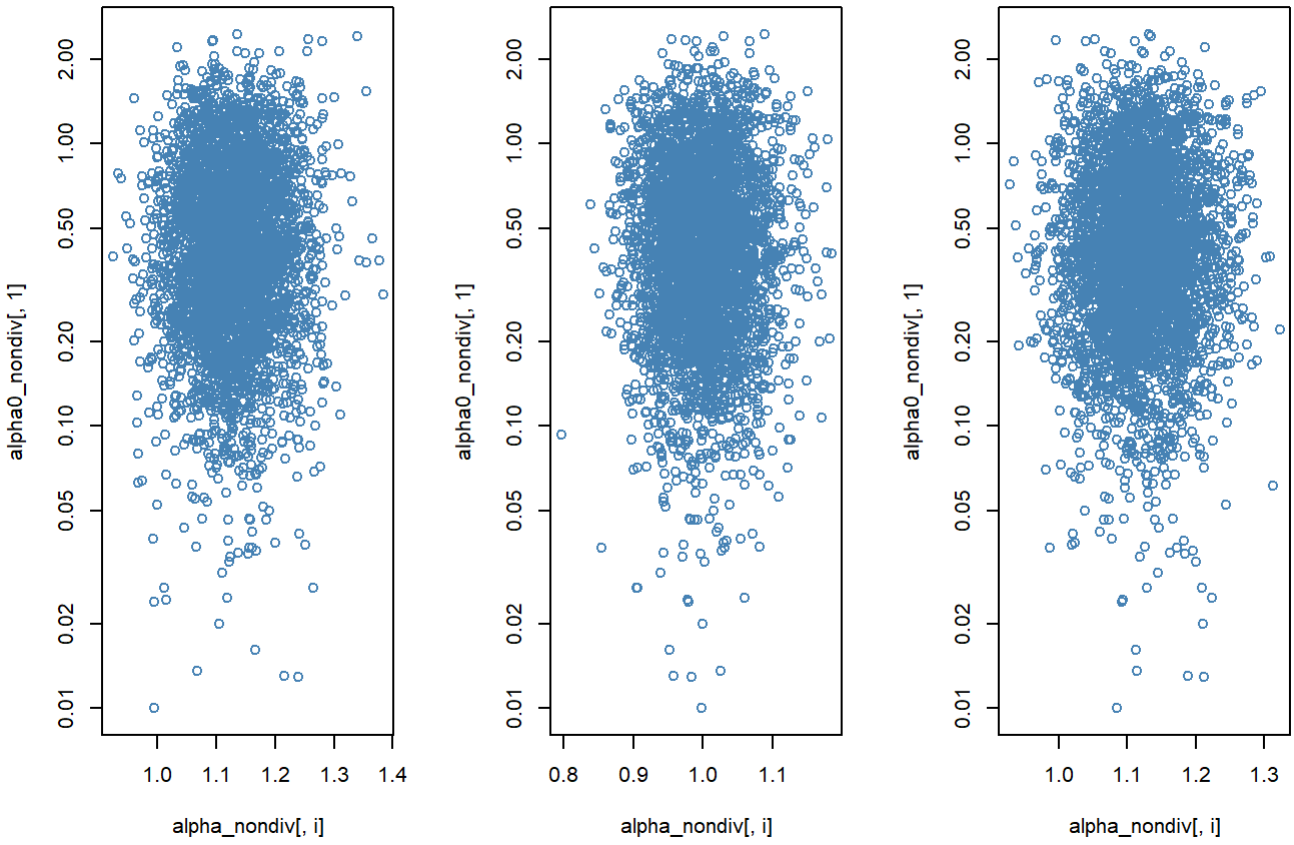

```
par(mfrow = c(1, 1))
```

```
par(mfrow = c(1, 3))

alpha_res <- split_diverging(weib_hier_reparam_fit, "alpha")
alpha_div <- alpha_res[[1]]
alpha_nondiv <- alpha_res[[2]]

alpha0_res <- split_diverging(weib_hier_reparam_fit, "alpha0")
alpha0_div <- alpha0_res[[1]]
alpha0_nondiv <- alpha0_res[[2]]

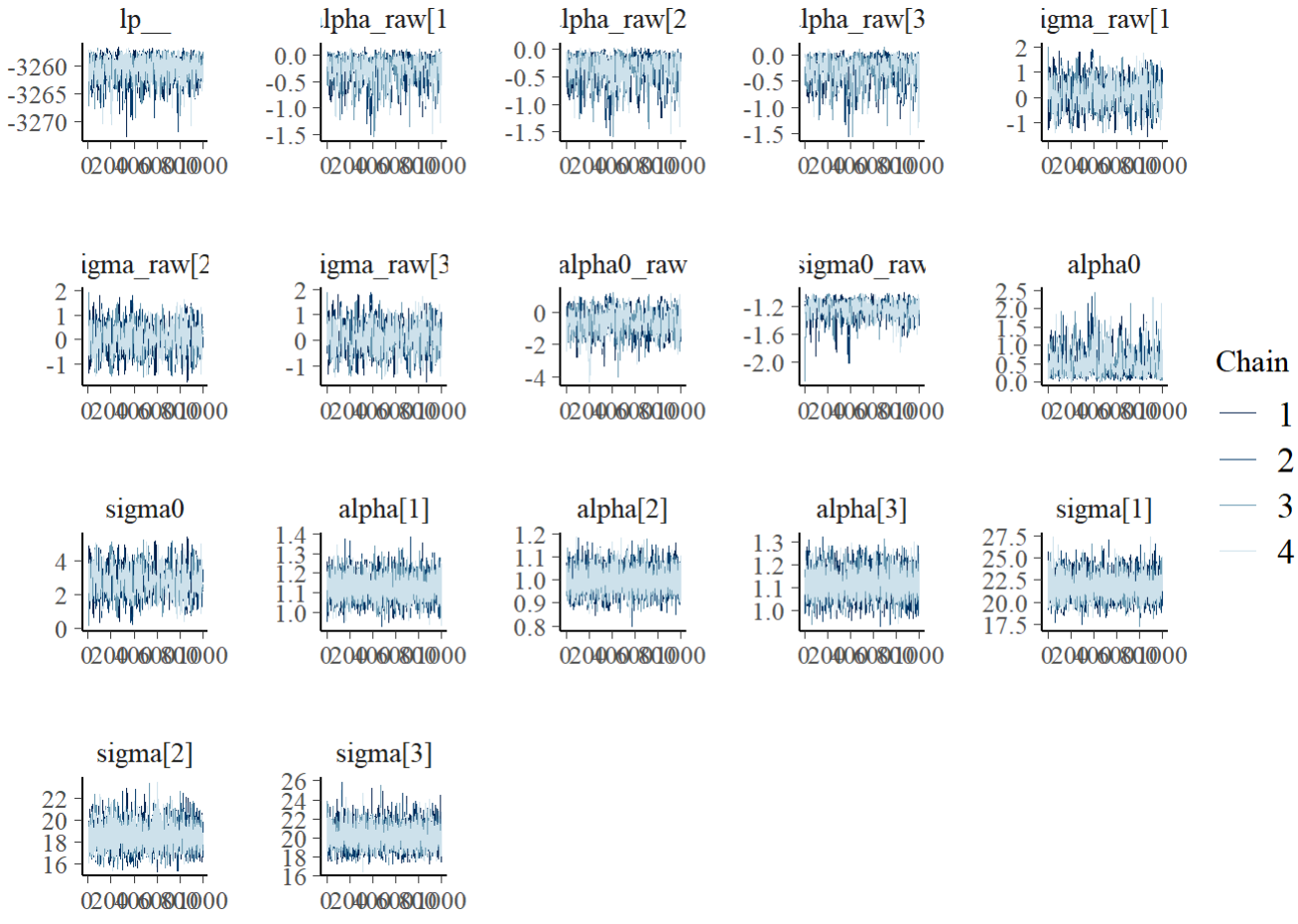
for (i in 1:3) {
    plot(alpha_nondiv[, i], alpha0_nondiv[, 1], col = "steelblue", log = "y")
    points(alpha_div[, i], alpha0_div[, 1], col = "lightgreen", pch = 19)
}
```



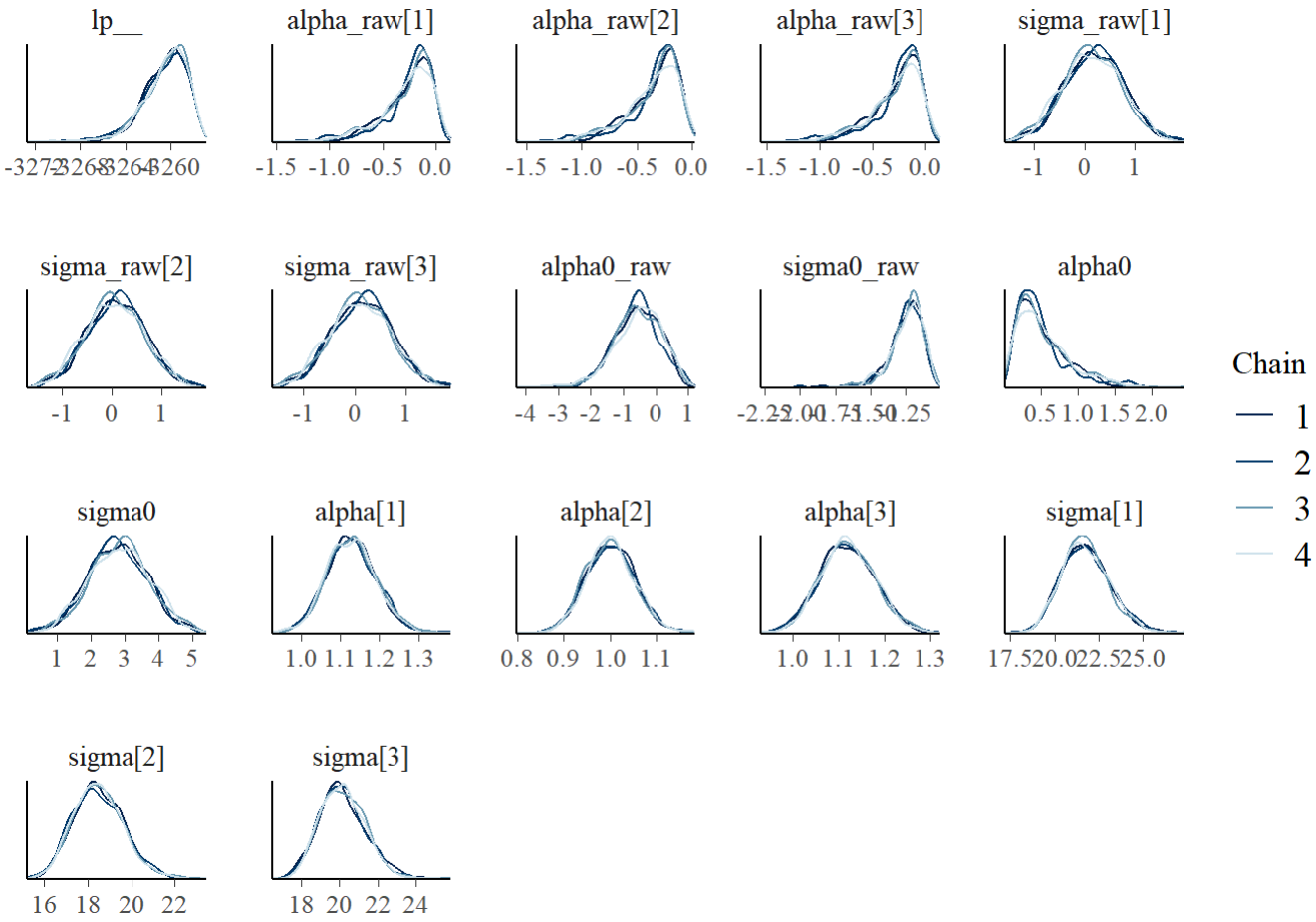

```
par(mfrow = c(1, 1))
```

```
posterior_draws_weib_reparam <- posterior::as_draws_df(weib_hier_reparam_fit)
```

```
mcmc_trace(posterior_draws_weib_reparam)
```



```
mcmc_dens_overlay(posterior_draws_weib_reparam)
```




```
t_range <- seq(0, 20, length.out = 1000)
cols <- c("steelblue", "orange", "forestgreen")

posterior_draws <- posterior::as_draws_df(weib_hier_reparam_fit)

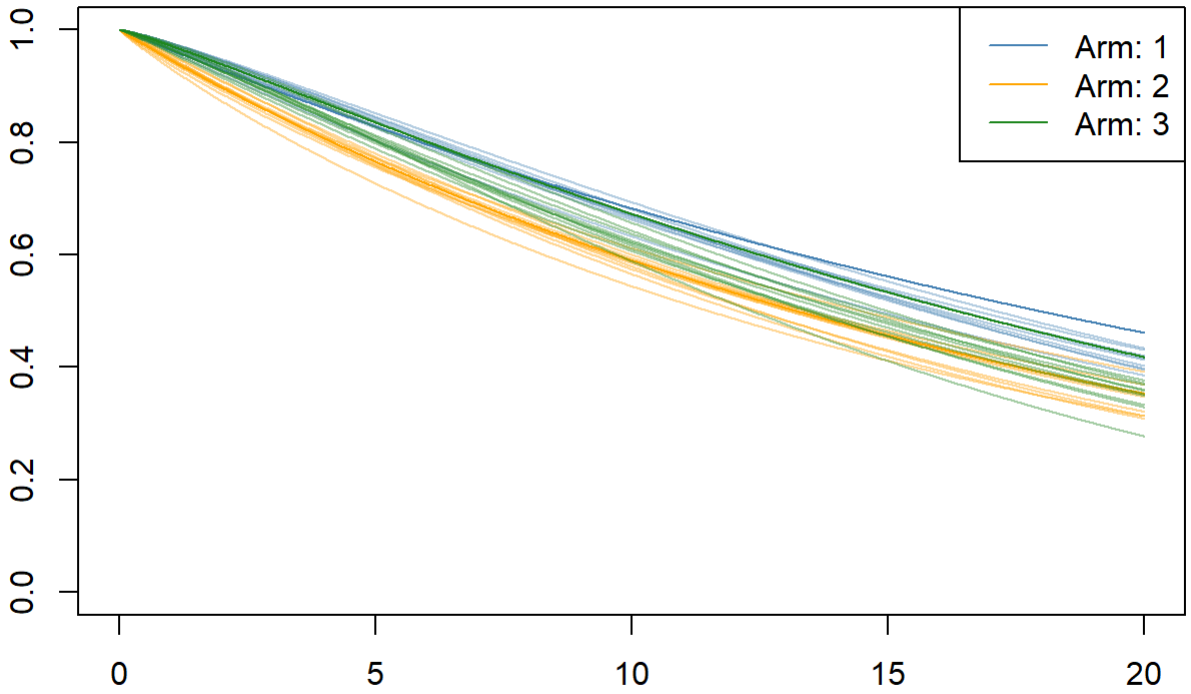
plot(NA, xlim = c(0, 20), ylim = c(0, 1), xlab = "", ylab = "")

for (i in 1:3) {
    alphas <- posterior_draws[[paste0("alpha[", i, "]")]]
    sigmas <- posterior_draws[[paste0("sigma[", i, "]")]]

    lines(t_range, exp(-(t_range / sigmas[1])^alphas[1]), col = cols[i])

    for (j in 1:10) {
        idx <- sample(length(alphas), 1)
        lines(t_range, exp(-(t_range / sigmas[idx])^alphas[idx]), col = adjustcolor(cols[i], alpha.f =
0.4))
    }
}

legend("topright", legend = c("Arm: 1", "Arm: 2", "Arm: 3"), col = cols, lty = 1)
```



Handling Missing Data

Censored data are a very particular kind of missing data. But we can handle them also in more general cases

Example 1: missing responses in a regression model

We generate a fictional dataset for this example…

```
n <- 200
p <- 100

X <- matrix(rnorm(n * p), nrow = n, ncol = p)
beta_true <- rnorm(p)
y_true <- X %*% beta_true + rnorm(n)
```

… and then split observed and missing data

```
obs_mask <- runif(n) > 0.5
obs_idx <- which(obs_mask)

y_obs <- y_true[obs_mask]
X_obs <- X[obs_mask, ]
X_mis <- X[!obs_mask, ]
```

We code and run the model **without** missing data

```
normal_reg <- "
data {
  int<lower=1> n;
  int<lower=1> p;
  vector[n] y;
  matrix[n, p] X;
}

parameters {
  real<lower=0> tau;
  real<lower=0> sigma;
  vector[p] eta;
  real alpha;
}

transformed parameters {
  vector[p] beta = eta * tau;
  vector[n] mean = alpha + X * beta;
}

model {
  alpha ~ normal(0, 10);
  eta ~ normal(0, 1);
  sigma ~ normal(0,1);
  tau ~ cauchy(0, 1);
  y ~ normal(mean, sigma);
}

generated quantities {
  vector[n] log_lik;
  for (i in 1:n)
      log_lik[i] = normal_lpdf(y[i] | mean[i], sigma);
}
"

stan_file <- "./stan/normal_reg.stan"
writeLines(normal_reg, stan_file)

normal_reg <- cmdstan_model(stan_file)
```

```
# Data dictionary for stan
normal_reg_data <- list(
    n = length(y_obs),
    p = ncol(X_obs),
    y = y_obs,
    X = X_obs
)

# MCMC run
normal_reg_fit <- normal_reg$sample(
    data = normal_reg_data,
    chains = 2,
    parallel_chains = 2,
    iter_warmup = 2000,
    iter_sampling = 2000,
    adapt_delta = 0.9,
    refresh = 1000,
    show_exceptions = FALSE,
    seed = 123
)
```

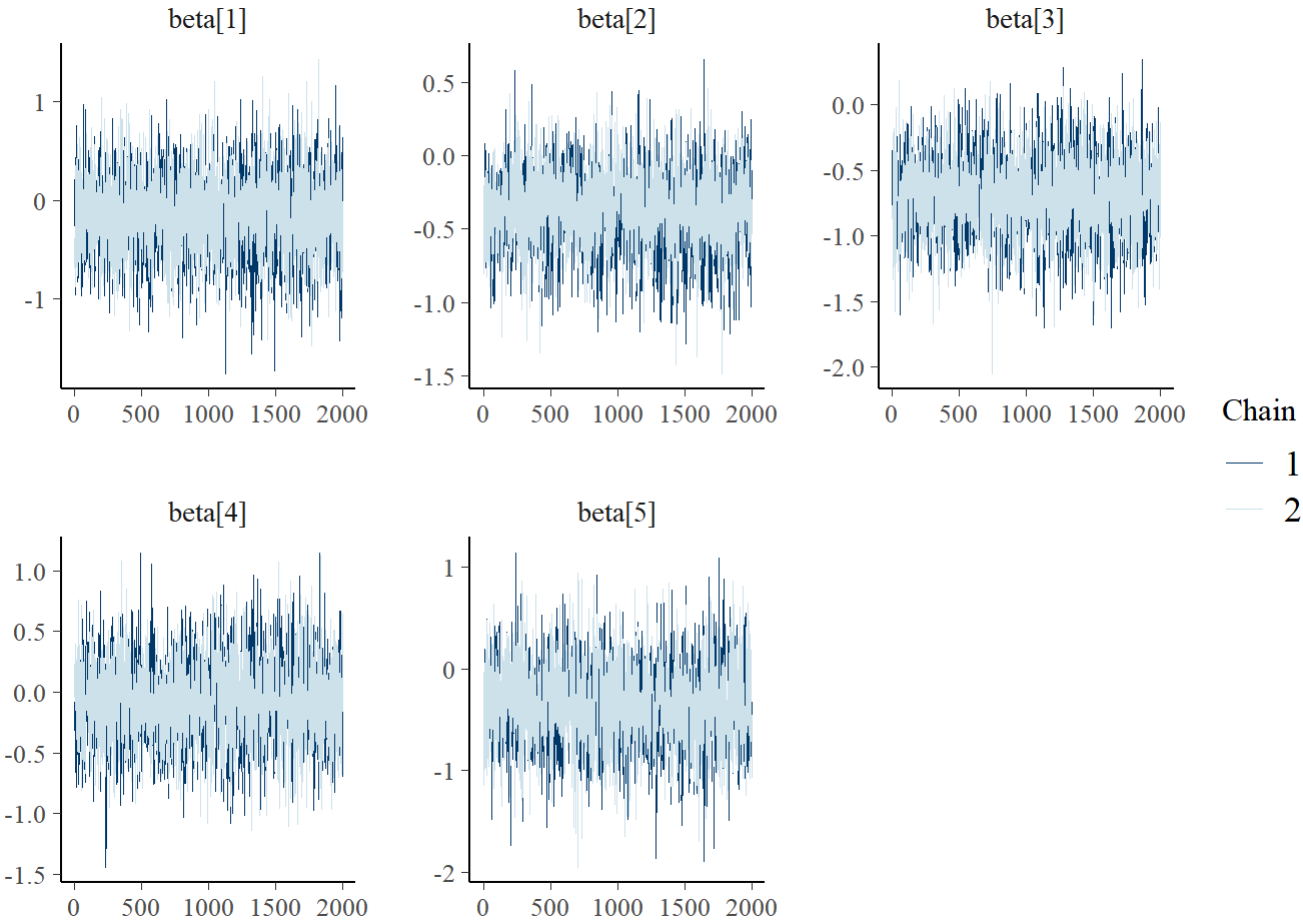
```
## Running MCMC with 2 parallel chains...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 3.9 seconds.
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 4.1 seconds.
##
## Both chains finished successfully.
## Mean chain execution time: 4.0 seconds.
## Total execution time: 4.2 seconds.
```

```
## Warning: 1 of 4000 (0.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.
```

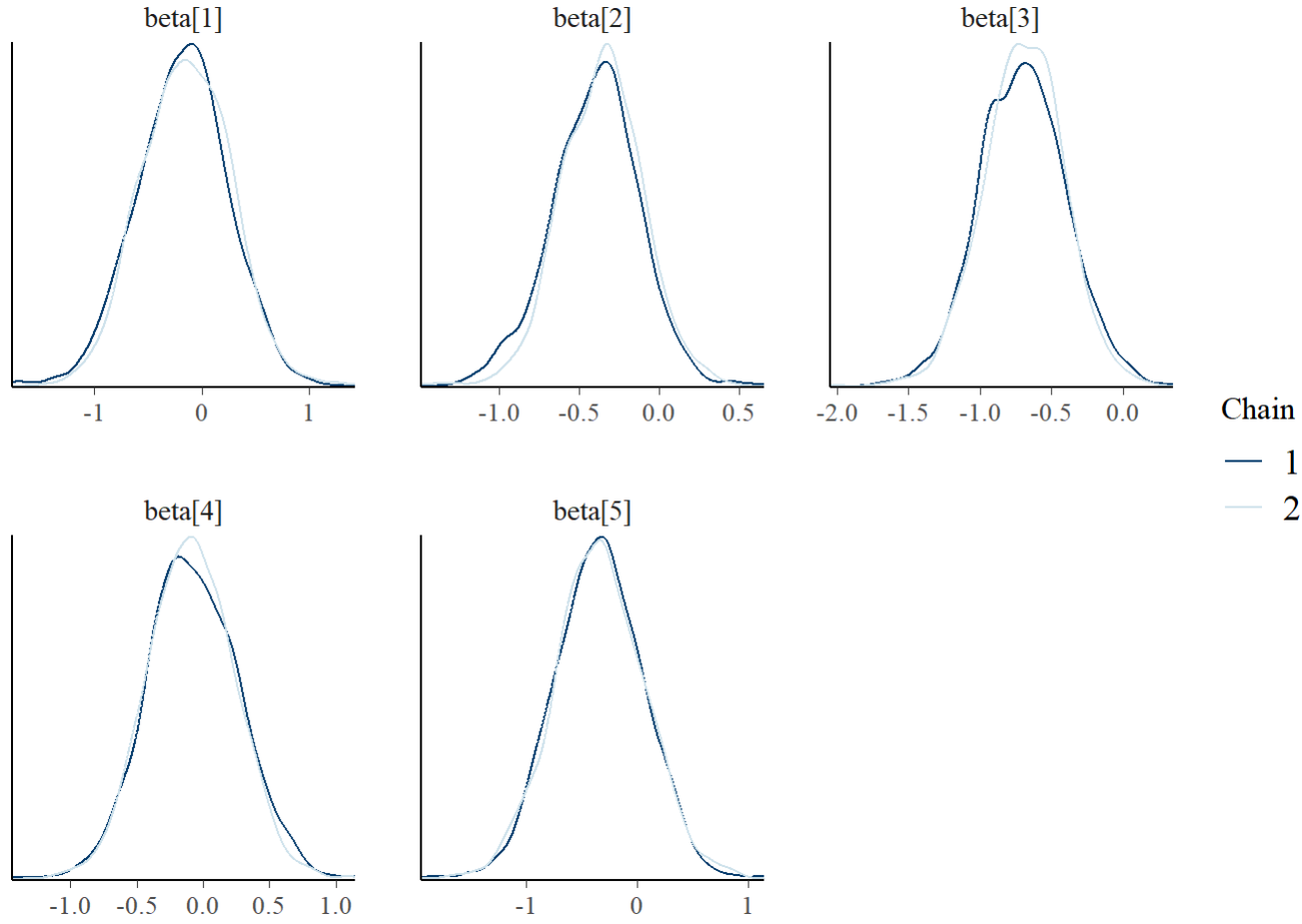
```
## Warning: 2 of 2 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

```
normal_reg_draws <- posterior::as_draws(normal_reg_fit)
```

```
mcmc_trace(normal_reg_draws, pars = c("beta[1]","beta[2]","beta[3]","beta[4]","beta[5]"))
```




```
mcmc_dens_overlay(normal_reg_draws, pars = c("beta[1]","beta[2]","beta[3]","beta[4]","beta[5]"))
```



And now the one in which we also impute the missing data

```
normal_reg_miss <- "
data {
  int<lower=1> n_obs;
  int<lower=1> n_mis;
  int<lower=1> p;
  vector[n_obs] y_obs;
  matrix[n_obs, p] X_obs;
  matrix[n_mis, p] X_mis;
}

parameters {
  real<lower=0> tau;
  real<lower=0> sigma;
  vector[p] eta;
  real alpha;
  vector[n_mis] y_mis;
}

transformed parameters {
  vector[p] beta = eta * tau;
  vector[n_obs] mean_obs = alpha + X_obs * beta;
  vector[n_mis] mean_mis = alpha + X_mis * beta;
}

model {
  alpha ~ normal(0, 10);
  eta ~ normal(0, 1);
  sigma ~ normal(0, 1);
  tau ~ cauchy(0, 1);
  y_obs ~ normal(mean_obs, sigma);
  y_mis ~ normal(mean_mis, sigma);
}

generated quantities {
  vector[n_obs] log_lik;
  for (i in 1:n_obs)
      log_lik[i] = normal_lpdf(y_obs[i] | mean_obs[i], sigma);
}
"

# Write stan model to file
stan_file <- "./stan/normal_reg_miss.stan"
writeLines(normal_reg_miss, stan_file)

# Compile stan model
normal_reg_miss <- cmdstan_model(stan_file)
```

```
reg_data <- list(
    n_obs = length(y_obs),
    n_mis = nrow(X_mis),
    p = ncol(X_obs),
    y_obs = y_obs,
    X_obs = X_obs,
    X_mis = X_mis
)

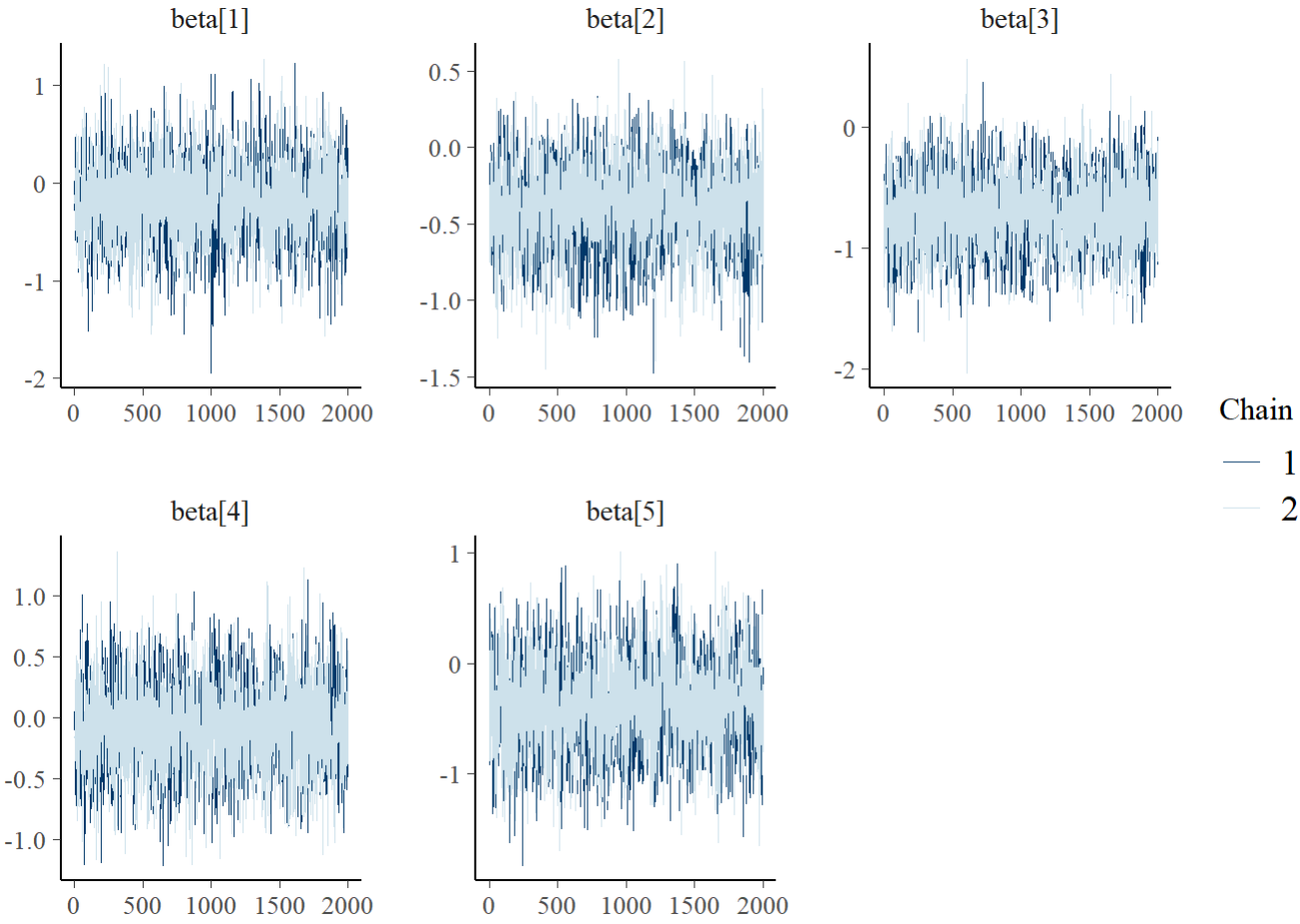
normal_reg_miss_fit <- normal_reg_miss$sample(
    data = reg_data,
    chains = 2,
    parallel_chains = 2,
    iter_warmup = 2000,
    iter_sampling = 2000,
    adapt_delta = 0.9,
    refresh = 1000,
    show_exceptions = FALSE,
    seed = 123
)
```

```
## Running MCMC with 2 parallel chains...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 24.6 seconds.
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 30.0 seconds.
##
## Both chains finished successfully.
## Mean chain execution time: 27.3 seconds.
## Total execution time: 30.1 seconds.
```

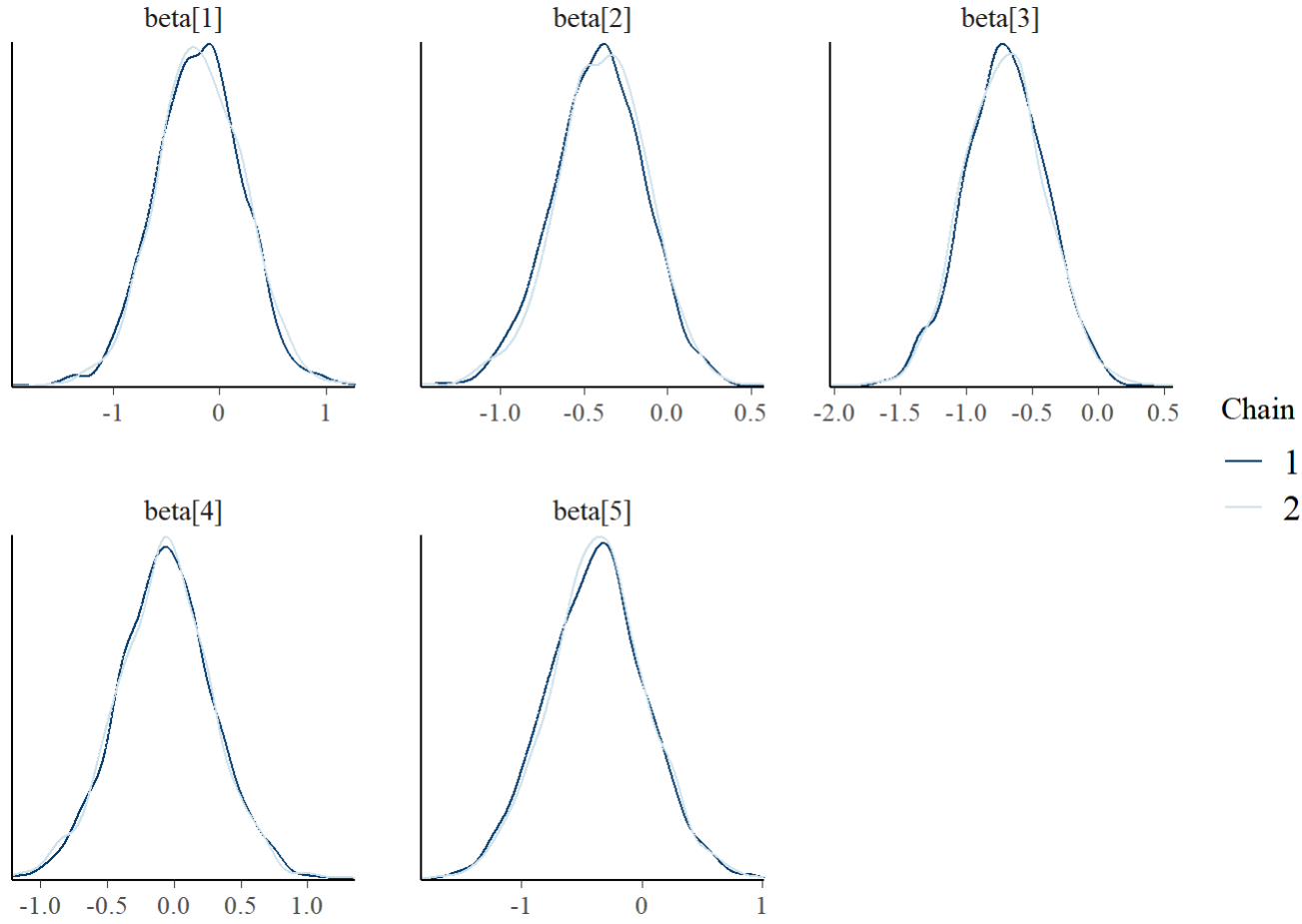
```
## Warning: 2 of 2 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

```
normal_reg_miss_draws <- posterior::as_draws(normal_reg_miss_fit)
```

```
mcmc_trace(normal_reg_miss_draws, pars = c("beta[1]","beta[2]","beta[3]","beta[4]","beta[5]"))
```




```
mcmc_dens_overlay(normal_reg_miss_draws, pars = c("beta[1]","beta[2]","beta[3]","beta[4]","beta[5]"))
```



Example 2: missing data in time series models

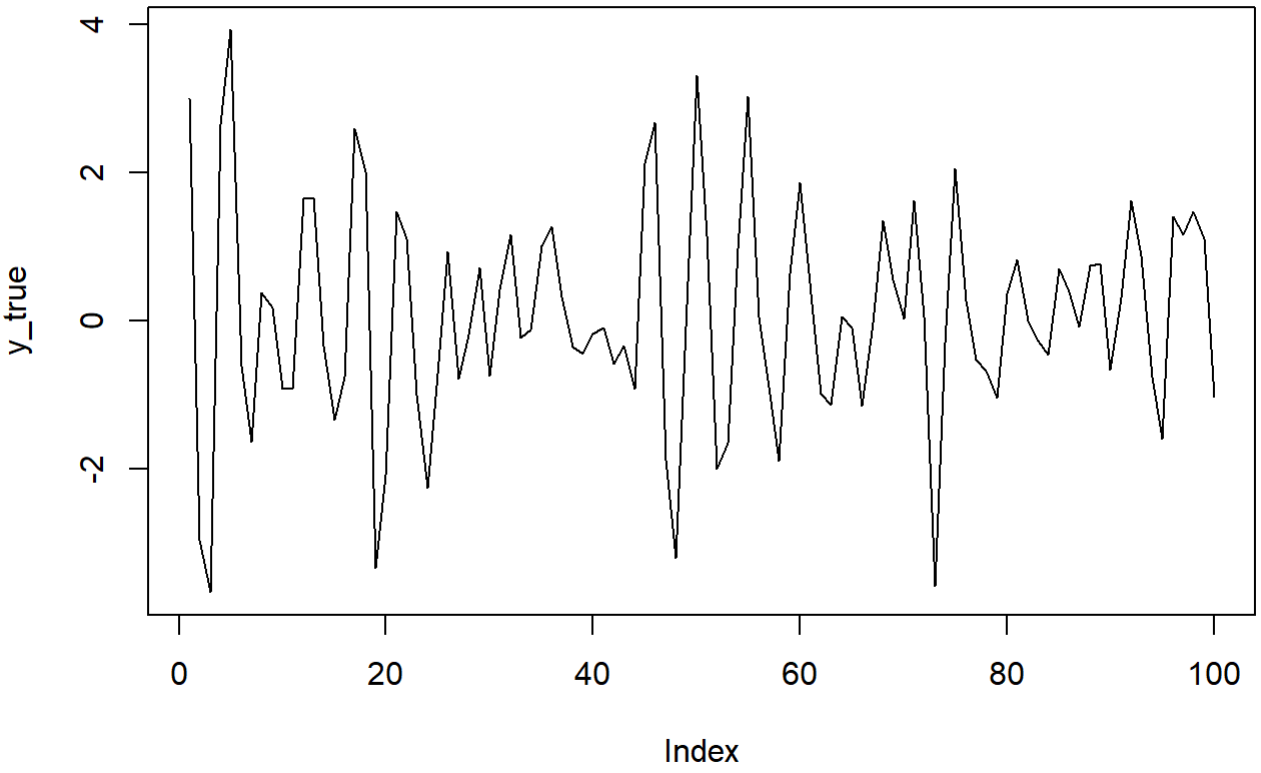
We again create a fictional dataset. Now missing and observed data are related!

```
phi <- c(-0.8, 0.35)
sig <- 1.0

y_true <- numeric(100)
y_true[1] <- 3
set.seed(123)
y_true[2] <- rnorm(1, mean = phi[1] * y_true[1], sd = sig)

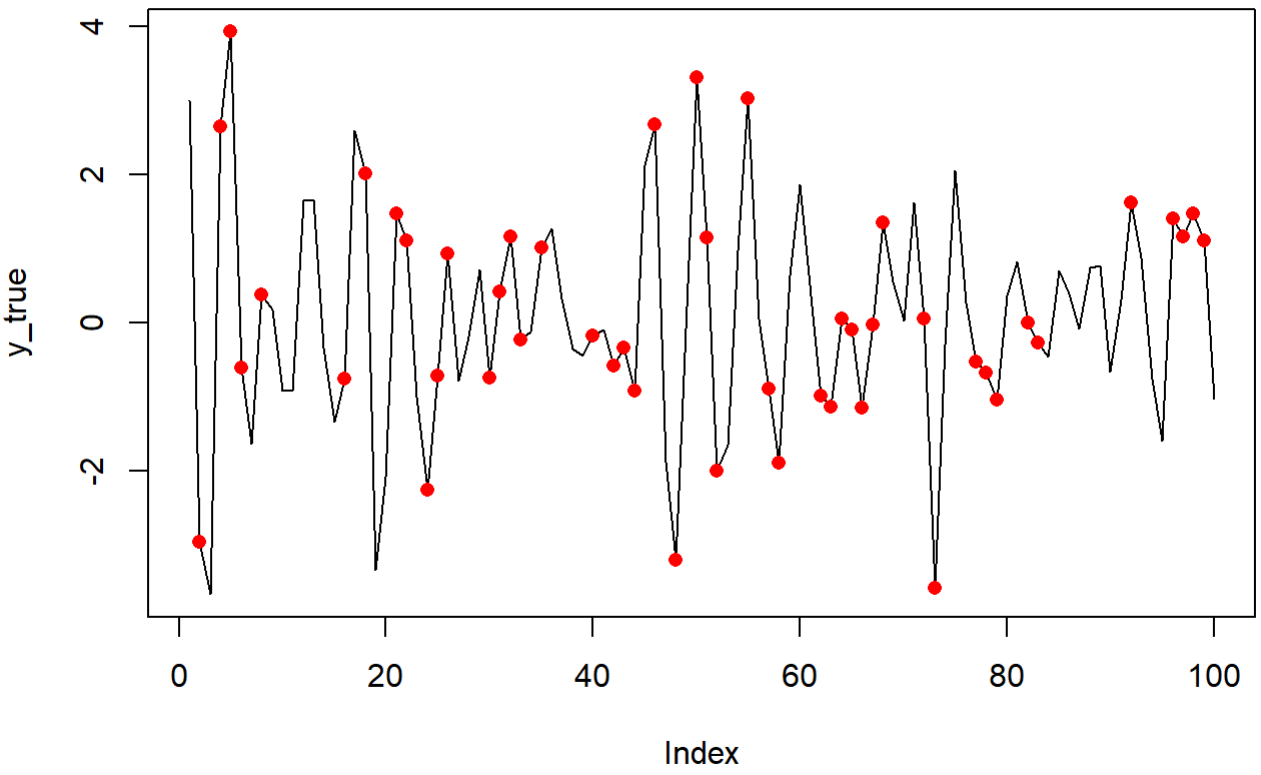
for (i in 3:100) {
    y_true[i] <- rnorm(1, mean = sum(y_true[(i-2):(i-1)] * phi), sd = sig)
}

plot(y_true, type = "l")
```




```
obs_mask <- runif(length(y_true)) > 0.5
obs_idx <- which(obs_mask)
miss_idx <- which(!obs_mask)

y_obs <- y_true[obs_mask]
plot(y_true, type = "l")
points(obs_idx, y_obs, col = "red", pch = 16)
```


y_true
4
2
0
-2
0
20
40
60
80
100
Index

```
ts_miss <- "
data {
    int<lower=1> n_obs;
    int<lower=1> n_mis;
    vector[n_obs] y_obs;
    array[n_obs] int observed_idx;
    array[n_mis] int missing_idx;
}

transformed data {
    int n_tot = n_obs + n_mis;
}

parameters {
    vector[n_mis] y_mis;
    vector[2] phi;
    real<lower=0> sigma;
}

transformed parameters {
    vector[n_tot] y;
    for (i in 1:n_obs) {
        y[observed_idx[i]] = y_obs[i];
    }
    for (i in 1:n_mis) {
        y[missing_idx[i]] = y_mis[i];
    }
}

model {
    for (i in 3:n_tot) {
        y[i] ~ normal(y[i-1] * phi[1] + y[i-2] * phi[2], sigma);
    }
    sigma ~ student_t(3, 0, 5);
    phi ~ normal(0, 1.5);
}
"

stan_file <- "./stan/ts_miss.stan"
writeLines(ts_miss, stan_file)

ts_miss <- cmdstan_model(stan_file)
```

```
ts_miss_data <- list(
    n_obs = length(y_obs),
    n_mis = length(miss_idx),
    y_obs = y_obs,
    observed_idx = obs_idx,
    missing_idx = miss_idx,
    show_exceptions = FALSE
)

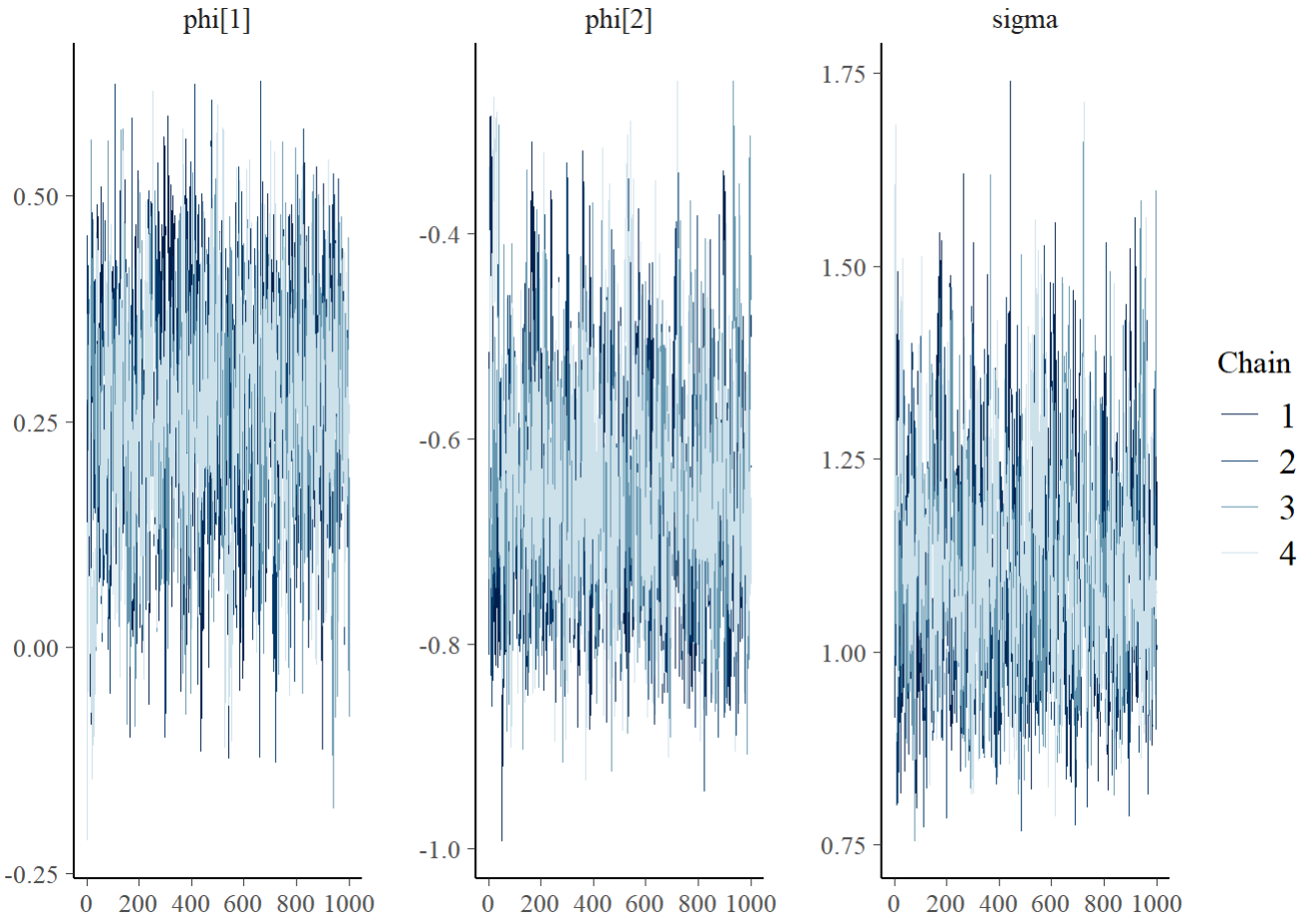
ts_miss_fit <- ts_miss$sample(data = ts_miss_data)
```

```
## Running MCMC with 4 sequential chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 1 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 1 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 1 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 1 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 1 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 1 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 1 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 1 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 1 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 1 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 1 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 0.4 seconds.
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 2 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 2 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 2 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 2 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 2 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 2 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 2 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 2 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 2 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 2 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 2 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 0.4 seconds.
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 3 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 3 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 3 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 3 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 3 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 3 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
```

```
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 3 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 3 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 3 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 3 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 3 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 0.4 seconds.
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 4 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 4 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 4 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 4 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 4 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 4 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 4 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 4 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 4 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 4 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 0.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 0.4 seconds.
## Total execution time: 2.0 seconds.
```

```
ts_miss_draws <- posterior::as_draws(ts_miss_fit)
```

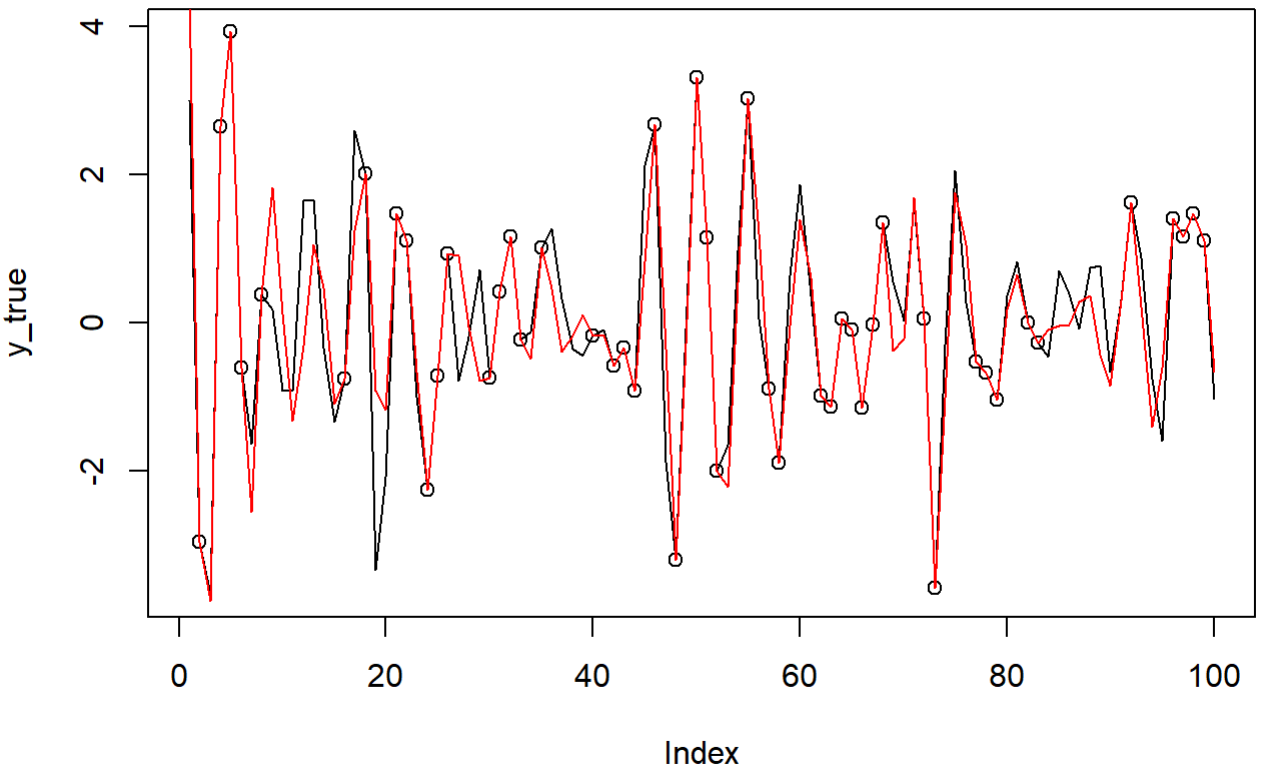
```
mcmc_trace(ts_miss_draws, pars = c("phi[1]", "phi[2]", "sigma"))
```




```
plot(y_true, type = "l")
points(obs_idx, y_obs)

y_draws <- posterior::subset_draws(ts_miss_draws, variable = "y")
y_mat <- posterior::as_draws_matrix(y_draws)
med_y <- apply(y_mat, 2, median)

lines(med_y, col = "red")
```


y_true
4
2
0
-2
0
20
40
60
80
100
Index